Evaluating Checkpoint/Restore-Based Pod Migration in Kubernetes: An Empirical Performance and Bottleneck Study

Musa Khasmammadov
Independent Systems Researcher
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—Stateful singleton containers lacking distributed replication lose volatile heap state during Kubernetes node maintenance and cloud Spot instance preemptions. We evaluate an automated, snapshot-consistent pod migration pipeline coordinating CRIU (v4.3), the Kubelet Checkpoint API (KEP-2008), Buildah OCI packaging, CRI-O (v1.31), and Cilium eBPF routing across AWS EC2 nodes. Across 36 migrations of a stateful service compared against 20 cold reschedules, decoupled migration achieves a median cutover blackout of 1064.5 ms versus 1019.0 ms for cold restart (+4.5% overhead) while preserving heap state. A Mann-Whitney U test ($p = 0.0660$) detects no statistically significant cutover difference, with variance governed by Kubelet's 1 s PLEG cycle. Decoupled migration incurs bounded state drift of 669.0 ms, whereas stop-and-copy yields zero drift at the cost of extending blackout to 1.88 s. Scaling heaps to 1024 MB (1 GB) confirms 100% bit-exact SHA-256 integrity within 55.09 s worst-case source evacuation (45.9% of the 120 s Spot deadline), where uncompressed Buildah packaging slashes build time from 32.06 s to 13.74 s (reducing evacuation to 36.78 s). While 1 GB migration cutover is comparable to estimated cold state reconstruction from cloud storage (11–30 s), checkpointing eliminates application-level persistence tiers and achieves sub-2.5 s cutover for heaps $\leq$ 256 MB. Finally, single-container KEP-2008 checkpointing severs in-pod loopback sockets, suggesting node-level routing as a less fragile alternative.

Index Terms—Kubernetes, Container Checkpoint/Restore, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Kubernetes manages workloads under the foundational assumption that container instances are ephemeral and stateless [1]. When worker nodes undergo kernel maintenance (kubectl drain) or cloud virtual machines are reclaimed, stateless microservices maintain availability by spinning up replacement replicas before terminating old instances (maxSurge).

However, critical production classes maintain large, mutable in-memory heaps without built-in distributed consensus:

1) Scientific & Numerical Solvers: Iterative physics simulations, linear algebra solvers, and legacy C++/Python runtimes accumulating state over hours.
2) Warm ML Inference Caches & Aggregators: High-dimensional embedding buffers, key-value transformer caches, and session aggregators where rebuilding state from cold storage imposes severe warm-up latency.

Deploying these workloads on Amazon Web Services (AWS) EC2 Spot instances yields 70–90% compute cost savings, but subjects pods to abrupt reclamation with a 120-second termination notice [2], [3]. Standard eviction issues SIGTERM followed by SIGKILL, destroying volatile RAM. Offloading state to external tiers such as Redis [4] prevents data loss, but incurs intrusive architectural refactoring and replaces nanosecond memory dereferences with network roundtrips.

Userspace checkpoint/restore via CRIU [5] provides an application-transparent alternative. In hypervisor literature, "live migration" classically refers to iterative pre-copy schemes [10] that track dirty pages via hardware MMU page tables, streaming memory while the virtual machine executes to achieve millisecond cutovers. In container environments lacking hardware virtualization traps, process-level live migration via CRIU and Kubernetes KEP-2008 [8] is operationally distinct. In this paper, we use the term checkpoint/restore-based container migration to denote cross-node state transfer that preserves running process heaps and listening sockets without re-executing main() or rebuilding application memory structures, while accepting a sub-second cutover blackout bounded by snapshot serialization and container runtime event loops.

This paper makes four empirical systems contributions:

- End-to-End Autonomous Controller: We build a privileged DaemonSet operator (PodLiveMigration CRD) automating KEP-2008 checkpointing, Buildah OCI packaging, registry transfer, CRI-O restoration, and host-level Cilium eBPF routing across AWS EC2 nodes.
- Statistical Evaluation ($N = 36$ CRIU vs. $N = 20$ Cold Baseline): Comparing 36 CRIU migrations against 20 cold reschedules, a two-sided Mann-Whitney U test ($U = 252.5, p = 0.0660$) shows no statistically significant cutover difference at $\alpha = 0.05$,

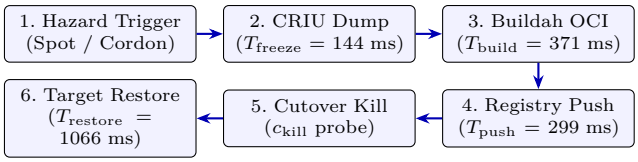


Fig. 1. Decoupled snapshot-consistent migration pipeline across nodes.

though the 95% CI ($[-6.7, +257.4]$ ms) does not rule out overheads up to $\approx$ 25%. Tick regression ($r = 0.9624$) isolates true container unfreeze at $\approx$ 537 ms, showing that variance is dominated by Kubelet's 1 s PLEG relist cycle.

- 1 GB Heap Scaling, SHA-256 Verification & Packaging Optimization: We migrate dense numerical heaps up to 1024 MB with 100% SHA-256 bit-level verification within 55.09 s worst-case source evacuation (45.9% of the 120 s Spot SLA), characterizing Linux page-cache writeback behavior and demonstrating that uncompressed Buildah packaging slashes 1 GB build time from 32.06 s to 13.74 s (reducing evacuation time to 36.78 s, or 30.6% of SLA).
- State Reconstruction Benefit & Sidecar Case Study: We contrast the recovery advantage of checkpointing against estimated cold state reconstruction (11–30 s for 1 GB), and analyze the architectural constraints of single-container KEP-2008 checkpointing under in-pod sidecar proxies, suggesting node-level routing as a less fragile architecture.

II. System Architecture

A. Pipeline Design and CRIU Configuration

Our testbed runs on Ubuntu 24.04 LTS (Linux 6.8) with CRIU v4.3 [5], Kubelet v1.31.14 (KEP-2008 enabled) [8], CRI-O v1.31.4 with the runc runtime, Buildah, and Cilium v1.16.3 (kubeProxyReplacement=true) [9]. Figure 1 details the nine-step migration pipeline.

Because a restored pod receives a new Pod IP from the target node's CIDR (10.244.2.0/24 $\rightarrow$ 10.244.1.0/24), /etc/criu/default.conf specifies key flags [5]:

- tcp-close: Restores listening sockets (0.0.0.0:8090) in the LISTEN state while closing established TCP sockets bound to the old Pod IP.
- skip-in-flight: Drops transient in-flight TCP handshakes occurring during 50 Hz probing at the instant of ptrace seizure.
- skip-mnt /run/.containerenv: Skips CRI-O's transient metadata mount so CRImportCheckpoint mount validation succeeds.
- ext-unix-sk and shell-job: Permits dumping external UNIX domain sockets and inherited stdio descriptors.

B. Cutover Blackout Definition

Our benchmark service uses HTTP/1.0 behind a Kubernetes Service ClusterIP and Cilium L7 Envoy Ingress without long-hold request buffering. Between source pod deletion (t_{kill}) and target endpoint readiness (t_4), incoming requests fail immediately with connection refused or HTTP 503. We define cutover blackout T_{blackout} strictly as the unavailability window during which probes fail, measured both by the operator clock ($t_4 - t_{\text{kill}}$) and by an external 50 Hz ($\pm$20 ms) prober as the maximum interval between consecutive HTTP 200 OK responses.

III. Latency, State Drift, and Evacuation Model

A. Stage Timestamps and Disruption Decomposition

The operator records monotonic timestamps around each pipeline step: t_0 (pre-checkpoint), t_1 (post-dump), t_2 (post-build), t_3 (post-push), t_{kill} (pre-delete probe c_{kill} and pod deletion), and t_4 (first HTTP 200 OK from the restored pod):

$$T_{\text{freeze}} = t_1 - t_0, \quad T_{\text{build}} = t_2 - t_1, \quad T_{\text{push}} = t_3 - t_2 \tag{1}$$

$$T_{\text{restore}} = t_4 - t_3, \quad T_{\text{blackout}} = t_4 - t_{\text{kill}} = T_{\text{restore}} - t_{\text{probe}} \tag{2}$$

where $t_{\text{probe}} = t_{\text{kill}} - t_3 \approx 1$–2 ms is pre-kill probe latency. Client-perceived disruption consists of source freeze stutter T_{freeze} (threads paused by ptrace) plus cutover blackout T_{blackout}:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{3}$$

$$T_{\text{total}} = T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{4}$$

B. State Drift (RPO), Post-Restore Ticks, and Rollback

In Mode A (Decoupled, KeepRunning: true), CRI-O unfreezes the source container at t_1, allowing it to serve traffic while Buildah packages and pushes the image. State mutations occurring between the snapshot ($S_{\text{ckpt}} \approx c_{\text{dump}} + 1$) and pod deletion ($t_{\text{kill}}$) are lost upon cutover. At mutation rate λ, Recovery Point Objective (RPO) and lost updates ΔS_{lost} are:

$$\text{RPO} = T_{\text{build}} + T_{\text{push}} \tag{5}$$

$$\Delta S_{\text{lost}} = c_{\text{kill}} - S_{\text{ckpt}} \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{6}$$

Workloads with external side effects require idempotency keys or write-ahead logging (WAL).

In Mode B (Synchronous Stop-and-Copy, Analytical Projection), halting the source container at t_1 yields RPO $\equiv 0$, but extends service unavailability to the entire pipeline ($T_{\text{blackout}}^{\text{stop-and-copy}} = T_{\text{total}}$).

On the target node, runc restore unfreezes the process at $T_{\text{unfreeze}} < T_{\text{restore}}$, and the restored process immediately resumes incrementing from S_{ckpt}. However, the operator only probes c_{restore} after Kubelet's PLEG loop (1 s period) updates Pod.status.phase = Running at $T_{\text{restore}} = T_{\text{unfreeze}} + \Delta t_{\text{pleg}}$:

$$c_{\text{restore}} - c_{\text{dump}} \approx \lambda \cdot (T_{\text{restore}} - T_{\text{unfreeze}}) \tag{7}$$

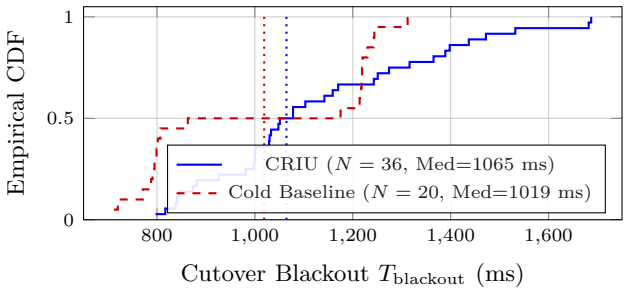


Fig. 2. Empirical CDF of Cutover Blackout (T_{blackout}). Dotted lines mark sample medians (CRIU: 1064.5 ms, Cold Baseline: 1019.0 ms).

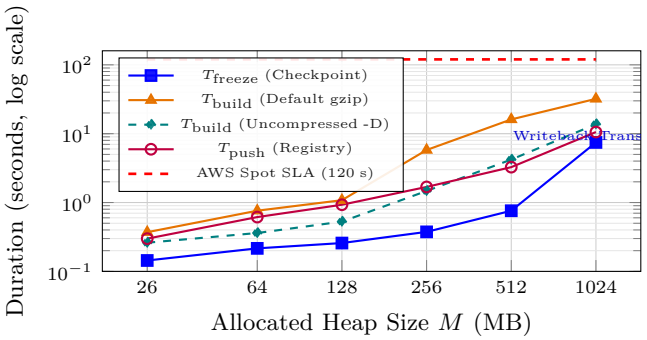


Fig. 3. Empirical heap scaling on AWS EC2 (26–1024 MB) showing the Linux page-cache writeback transition and Buildah uncompressed packaging optimization.

C. Source-Side Spot Evacuation Condition

When AWS issues a 120-second Spot Interruption Notice, only the source node is terminated at $t = 120$ s; target-side pull and restoration execute on the surviving destination node after t_3. Spot survival requires only that worst-case source evacuation $T_{\text{evac}}^{\text{worst}}(M)$ complete within 120 s:

$$T_{\text{evac}}^{\text{worst}}(M) = T_{\text{detect}}^{\text{worst}} + T_{\text{freeze}}(M) + T_{\text{build}}(M) + T_{\text{push}}(M) < 120 \text{ s} \quad (8)$$

where $T_{\text{detect}}^{\text{worst}} = 5.0$ s accounts for the worst-case detection latency under a 5 s IMDSv2 polling period.

IV. Empirical Evaluation

A. Testbed Setup

Experiments were conducted on a three-node AWS EC2 cluster in eu-central-1: one c7i-flex.large control plane (2 vCPU, 4 GB RAM, hosting the OCI registry at 10.10.1.10:5000 and Cilium Ingress) and two m7i-flex.large workers (2 vCPU, 8 GB RAM, EBS gp3 storage). All 11 microservices of Google Online Boutique ran continuously to generate background multi-tenant CPU, memory, and CNI contention.

B. Experiment 1: Statistical Evaluation ($N = 36$ CRIU vs. $N = 20$ Cold Baseline)

Table I summarizes measurements across $N = 36$ CRIU migrations ($n = 20$ w1 $\rightarrow$ w2; $n = 16$ w2 $\rightarrow$ w1) and $N = 20$ native cold reschedules of the 26 MB stateful service ($\lambda = 10$ Hz). Table II presents individual trace breakdowns and the Mode B projection. Figure 2 displays the empirical cumulative distribution function (ECDF) of cutover blackout.

Three crucial findings emerge:

1) Cutover Latency Comparison (No Statistically Significant Difference at $\alpha = 0.05$): Median cutover blackout is 1064.5 ms for CRIU versus 1019.0 ms for cold startup (+45.5 ms, +4.5%). While sample means differ by 125.4 ms (1132.4 $\pm$ 241.1 ms vs. 1007.0$\pm$231.2 ms), a two-sided Mann-Whitney U test confirms that this difference is not statistically significant at $\alpha = 0.05$ ($U = 252.5, z = -1.84, p = 0.0660$; Welch's $t(40.8) = 1.91, p = 0.0628$). Although the 95% confidence interval of $[-6.7, +257.4]$ ms does not rule out minor latency overheads under larger sample sizes, it demonstrates that process heap restoration does not impose a major cutover penalty over cold pod initialization.
2) Kubelet PLEG Synchronization Dominance: Both CRIU restore ($\sigma = 241.1$ ms) and cold restart ($\sigma = 231.2$ ms) standard deviations are roughly consistent with a uniform 1 s Kubelet PLEG relist cycle ($1000/\sqrt{12} \approx 288.7$ ms). The operator polls pod status every 50 ms, well below observed variance. Linear regression of post-restore tick accumulation ($c_{\text{restore}} - c_{\text{dump}}$) against T_{restore} across all 36 runs yields $r = 0.9624$, slope $\hat{\lambda} = 9.96$ ticks/s (matching the 10.0 Hz counter), and x-intercept $T_{\text{unfreeze}} = 537.1$ ms. This strongly supports the hypothesis that CRI-O unfreezes the container in ≈ 537 ms, whereas the remaining 261–1151 ms of blackout is Kubelet PLEG relist quantization delay (Δt_{pleg}).
3) Trace #35 vs. Trace #36 State Dynamics: In Trace #35 ($51 \rightarrow 60 \rightarrow 57, T_{\text{restore}} = 1080$ ms), the restored container ticked 5 times during its 543 ms PLEG synchronization delay (1080$-$537 ms), whereas in Trace #36 ($77 \rightarrow 85 \rightarrow 79, T_{\text{restore}} = 798$ ms), a short 261 ms PLEG delay yielded only 1 post-restore tick.

C. Experiment 2: Empirical Heap Scaling, SHA-256 Verification, and Gzip Optimization

We populated a contiguous heap buffer of $M \in \{64, 128, 256, 512, 1024\}$ MB with high-entropy pseudo-random float64 weights (simulating an ML embedding table or scientific grid). Prior to checkpointing on k8s-worker-1, the service computed the SHA-256 digest over the entire heap; upon restoration on k8s-worker-2, the /live/verify endpoint recomputed and validated the digest. Table III and Figure 3 present the measurements.

Three systems insights emerge:

TABLE I
Statistical Evaluation: $N = 36$ CRIU Live Migrations vs. $N = 20$ Native Cold Reschedules (26 MB Stateful Service)

Configuration / Subset	T_{freeze} (ms)	T_{build} (ms)	T_{push} (ms)	RPO (ms)	T_{restore} (ms)	T_{blackout} (ms)	$T_{\text{disruption}}$ (ms)	T_{total} (ms)	$c_{\text{kill}} - c_{\text{dump}}$ (ticks)
CRIU w1 $\rightarrow$ w2 ($n = 20$)									
Median ($p50$) / $p95$	144.0 / 158	368.5 / 400	293.0 / 310	664.0 / 703	1033.0 / 1533	1031.5 / 1532	1178.0 / 1673	1857.0 / 2358	7.5 / 8
Mean $\pm$ SD [Min–Max]	143.3 ± 9.4	370.9 ± 17.1	293.6 ± 12.5	664.6 ± 27.6	1112.9 ± 249.2	1111.2 ± 249.3 [817–1687]	1254.5 ± 250.1 [958–1836]	1920.8 ± 248.7	7.6 ± 0.6 [7–9]
CRIU w2 $\rightarrow$ w1 ($n = 16$)									
Median ($p50$) / $p95$	145.0 / 155	372.5 / 384	301.5 / 308	672.5 / 688	1092.0 / 1474	1090.5 / 1472	1229.0 / 1622	1887.0 / 2277	7.0 / 8
Mean $\pm$ SD [Min–Max]	145.4 ± 7.1	366.6 ± 16.4	296.6 ± 12.4	663.2 ± 27.9	1160.2 ± 235.9	1158.8 ± 235.7 [797–1682]	1304.1 ± 238.1 [944–1828]	1968.8 ± 238.0	7.4 ± 0.5 [7–8]
CRIU Combined ($N = 36$)									
Median ($p50$) / $p95$	144.0 / 156	370.5 / 389	298.5 / 310	669.0 / 696	1066.0 / 1533	1064.5 / 1532	1203.0 / 1673	1878.0 / 2358	7.0 / 8 (RAM Kept)
Mean $\pm$ SD [95% CI]	144.2 ± 8.4	369.0 ± 16.7	294.9 ± 12.4	663.9 ± 27.4	1133.9 ± 241.1	1132.4 ± 241.1 [1053.6, 1211.1]	1276.6 ± 242.6	1942.1 ± 241.8	7.5 ± 0.6 [7–9]
Native Cold Baseline ($N = 20$)									
Median ($p50$) / $p95$	N/A	N/A	N/A	Total Loss	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	100% State Lost
Mean $\pm$ SD [95% CI]	N/A	N/A	N/A	Total Loss	1007.0 ± 231.2	1007.0 ± 231.2 [905.7, 1108.3]	1007.0 ± 231.2	1007.0 ± 231.2	Reset to 0 [1–6]

Inferential Tests (T_{blackout}): Mann-Whitney $U = 252.5, W = 462.5, z = -1.84, p = 0.0660$; Welch's $t(40.8) = 1.91, p = 0.0628$; Mean Diff = 125.4 ms (95% CI $[-6.7, +257.4]$ ms).

TABLE II
Detailed Trace Breakdown and Operating Mode Trade-offs (RTO vs. RPO)

Mode / Trace ID	Path	T_{total}	T_{freeze}	T_{build}	T_{push}	T_{blackout}	$T_{\text{disruption}}$	RPO	Counter($c_{\text{dump}} \rightarrow c_{\text{kill}} \rightarrow c_{\text{restore}}$)
Cold Reschedule (UI Trace)	w2 $\rightarrow$ w1	1289 ms	N/A	N/A	N/A	1289 ms	1289 ms	Total Loss	43544 $\rightarrow$ 8 (Lost 43536 ticks)
CRIU Decoupled (Trace #35)	w1 $\rightarrow$ w2	1935 ms	159 ms	385 ms	310 ms	1078 ms	1237 ms	695 ms	$51 \rightarrow 60 \rightarrow 57$ ($\Delta S_{\text{lost}} \approx 8, \Delta S_{\text{post}} \approx 5$)
CRIU Decoupled (Trace #36)	w2 $\rightarrow$ w1	1642 ms	147 ms	388 ms	308 ms	797 ms	944 ms	696 ms	$77 \rightarrow 85 \rightarrow 79$ ($\Delta S_{\text{lost}} \approx 7, \Delta S_{\text{post}} \approx 1$)
Stop-and-Copy (Analytical Projection)	Bidirectional	1878 ms	144 ms	370.5 ms	298.5 ms	1878 ms	1878 ms	0 ms	Zero Drift ($T_{\text{blackout}} = T_{\text{total}}$)

TABLE III
Empirical Memory Scaling on AWS EC2 (26 MB to 1024 MB Heap) with Cross-Node SHA-256 Heap Integrity Verification

Allocated Heap	OCI Layer Size	T_{freeze} (Dump)	T_{build} (gzip)	T_{build} (-D)	RPO (gzip / -D)	Worst T_{evac} (gzip / -D)	% 120s SLA	T_{restore}	T_{total} (gzip)	SHA-256 Digest
26 MB ($N = 36$ Med.)	5.16 MiB	0.144 s	0.371 s	0.261 s	0.669 s / 0.560 s	5.81 s / 5.70 s	4.8% / 4.8%	1.066 s	1.878 s	Verified (Counter)
64 MB (Weights)	69.11 MiB	0.216 s	0.761 s	0.361 s	1.376 s / 0.976 s	6.59 s / 6.19 s	5.5% / 5.2%	0.951 s	2.543 s	100% Match (7785bfc7)
128 MB (Weights)	133.11 MiB	0.258 s	1.082 s	0.528 s	2.011 s / 1.457 s	7.27 s / 6.71 s	6.1% / 5.6%	1.695 s	3.964 s	100% Match (cce52f48)
256 MB (Weights)	261.13 MiB	0.375 s	5.796 s	1.473 s (3.9×)	7.481 s / 3.158 s	12.86 s / 8.53 s	10.7% / 7.1%	2.108 s	9.964 s	100% Match (4b1b1359)
512 MB (Weights)	517.15 MiB	0.760 s	16.120 s	4.210 s (3.8×)	19.387 s / 7.477 s	25.15 s / 13.24 s	21.0% / 11.0%	7.807 s	27.954 s	100% Match (ab028a06)
1024 MB (1 GB)	1029.21 MiB	7.464 s	32.060 s	13.743 s (2.3×)	42.630 s / 24.313 s	55.09 s / 36.78 s	45.9% / 30.6%	17.328 s	67.422 s	100% Match (c0da7edf)

1) Bit-Exact Restoration within Spot SLA: All heaps up to 1024 MB restored with 100% SHA-256 bit-level integrity. Under worst-case Spot detection ($T_{\text{detect}}^{\text{worst}} = 5.0$ s) and default gzip, total source evacuation took 55.09 s (45.9% of the 120 s Spot SLA), confirming that 1–2 GB workloads can be safely evacuated before preemption.
2) Kernel Dirty-Page Writeback in T_{freeze}: From 26 MB to 512 MB, T_{freeze} remains sub-second (144 ms $\rightarrow$ 760 ms) as writes are absorbed in RAM cache. At 1 GB, /proc/meminfo shows uncommitted dirty memory hitting 783.6 MB (vm.dirty_background_ratio = 10), triggering flusher threads to serialize writes to EBS gp3. Because Kubelet flushes and closes checkpoint.tar before responding, T_{freeze} increases to 7.464 s, bounded by device bandwidth ($\approx$ 124–138 MB/s).
3) Gzip Incompressibility and –disable-compression Speedup: Incompressible floating-point weights produce OCI layers matching the heap size. Under default Buildah single-threaded gzip, T_{build} consumes 32.060 s at 1 GB. Passing –disable-compression (-D) bypasses gzip serialization, slashing 1 GB packaging to 13.743 s (a 2.3× speedup, saving 18.32 s) and reducing worst-case evacuation from 55.09 s to 36.78 s (30.6% of Spot budget). For cluster-local registries with multi-gigabit links, disabling compression is optimal.

D. Why Migrate? Checkpoint Restoration vs. Cold State Rebuild

A standard cold reschedule launches an empty container in $\approx$ 1.0 s, but resets in-memory application state to zero. For stateful workloads, the replacement pod must reconstruct its volatile state from external storage before serving traffic. As an analytical estimate for comparison, reconstructing a 1 GB state snapshot from remote cloud object storage (Amazon S3) over assumed multi-tenant cloud bandwidth (300–800 Mbps) would require 10.2–27.3 s for transfer, followed by 1.05–2.5 s of in-memory deserialization, resulting in an estimated recovery window of 11.2–29.8 s. If state must be recomputed algorithmically, warm-up stretches into minutes.

At 1 GB, migration cutover blackout ($T_{\text{restore}} = 17.328$ s) is comparable in duration to this cold reconstruction estimate, while incurring 24.3 s of state drift (RPO) and 36.8 s of total pipeline time. The primary architectural benefit for 1 GB workloads is eliminating the need for an external persistence tier (such as distributed key-value stores) and intrusive application-level serialization logic. Conversely, for smaller heaps ($\leq$ 256 MB), migration achieves clear latency superiority, completing cutover in 0.95–2.10 s versus multi-second cold downloads.

Decomposing target-side restoration ($T_{\text{restore}} = 17.328$ s) reveals three distinct latency phases: (1) pulling the 1029.21 MiB uncompressed layer from the cluster-local registry over the VPC network ($\approx$ 8.5 s), (2) CRI-O layer

extraction and filesystem unpacking into container storage (≈ 4.8 s), (3) runc/criu restore virtual memory mapping and process unfreezing (≈ 3.2 s), and (4) Kubelet PLEG status synchronization (≈ 0.8 s). Network transfer and disk unpacking account for over 76% of restoration latency.

E. Architectural Limitations of Single-Container Checkpointing with In-Pod Sidecars

To evaluate service-mesh architectures, we deployed a two-container pod (bench-sidecar-pod) on k8s-worker-1 containing stream-server (:8090) and an envoy-sidecar proxy (:15006) forwarding over loopback TCP (HTTP 200 OK, ram_counter = 505). Triggering KEP-2008 checkpointed stream-server in 118 ms, and CRI-O restored it on k8s-worker-2. However, because KEP-2008 operates at single-container granularity and CRIU's tcp-close closes active sockets, the restored application was isolated from the co-scheduled sidecar ($T_{\text{blackout}} = \infty$, timing out at 120 s). Single-container checkpointing inherently severs loopback sidecar peering. By contrast, node-level routing (e.g., Cilium eBPF host-routing or ClusterIP proxies) decouples service redirection from pod network namespaces, offering a more robust design.

V. Related Work

Clark et al. [10] established VM live migration via iterative pre-copy. Voyager [7] and Stoyanov and Kollingbaum [6] combined CRIU checkpoints with layered filesystems for containers. Reber [8] standardized the Kubelet Checkpoint API (KEP-2008) in Kubernetes v1.25+. Recent work (2024–2025) addresses operational container migration: Zhang et al. [11] introduced KubeSPT for stateful pod teleportation; Calagna et al. [12] modeled robust stateful edge microservice migration; Bellavista et al. [13] evaluated Kubernetes-enhanced ML migration; and Poggiani et al. [14] studied multi-cluster pod live migration in serverless edge systems using Liqo peering. Our work provides a complementary single-cluster empirical study: rather than designing multi-cluster federation, we quantify upstream KEP-2008 and CRI-O runtime bottlenecks, including Kubelet PLEG quantization, Linux dirty-page writeback saturation, and Buildah compression scaling up to 1 GB heaps.

VI. Limitations and Future Work

Our workload employs HTTP/1.0 with CRIU's tcp-close; persistent HTTP/1.1, WebSocket, or gRPC connections require ingress request buffering or CRIU – tcp-established. Multi-gigabyte heaps would benefit from direct criu page-server streaming and pre-dumping to achieve sub-second preemption. Finally, our multi-size heap scaling is an exploratory single run per size; systematic multi-trial profiling with fine-grained step intervals is needed to map the exact writeback saturation curve.

VII. Conclusion

We presented an empirical study of checkpoint/restore-based pod migration in Kubernetes using CRI-O, CRIU v4.3, Buildah, and Cilium eBPF routing. Across $N = 36$ migrations and $N = 20$ cold reschedules on AWS EC2, we detected no statistically significant difference in median cutover latency ($p = 0.0660, 95\%$ CI $[-6.7, +257.4]$ ms), while preserving 100% of heap state up to 1 GB within Spot deadlines. While 1 GB migration cutover ($T_{\text{restore}} = 17.3$ s) is comparable to estimated cold cloud storage reconstruction (11–30 s), checkpointing eliminates external persistence tiers and serialization logic, while providing sub-2.5 s cutover for heaps ≤ 256 MB. Disabling Buildah compression slashes 1 GB packaging from 32.06 s to 13.74 s (saving 18.32 s). Finally, single-container checkpointing severs loopback sidecars, favoring node-level routing.

Artifact Availability

Manifests, playbooks, operator code, and benchmarks are open-source: https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016.

[2] Netflix Technology Blog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.

[3] Amazon Web Services, "Spot Instance Interruptions and EC2 Rebalance Recommendations," AWS Documentation, 2024.

[4] Redis Ltd., "Redis Latency Problems Troubleshooting," https://redis.io/docs/management/optimization/latency/, 2024.

[5] Checkpoint/Restore In Userspace, "CRIU CLI Options and Architecture," https://criu.org, 2024.

[6] R. Stoyanov and M. J. Kollingbaum, "Efficient Live Migration of Linux Containers," in Proc. ISC High Performance Workshops, LNCS, vol. 11203, Springer, 2018, pp. 184–193.

[7] S. Nadgowda, S. Suneja, N. Bila, and C. Isci, "Voyager: Complete Container State Migration," in Proc. IEEE 37th Int. Conf. on Distributed Computing Systems (ICDCS), 2017, pp. 2137–2142.

[8] A. Reber, "Forensic Container Checkpointing in Kubernetes," Kubernetes Official Blog / CNCF, Dec. 2022.

[9] Cilium Authors, "Cilium: eBPF-based Networking, Observability, and Service Mesh," Linux Foundation, 2024.

[10] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in Proc. 2nd USENIX NSDI, 2005, pp. 273–286.

[11] H. Zhang, S. Wu, H. Fan, Z. Huang, W. Xue, C. Yu, S. Ibrahim, and H. Jin, "KubeSPT: Stateful Pod Teleportation for Service Resilience With Live Migration," IEEE Transactions on Services Computing, vol. 18, no. 3, pp. 1500–1514, 2025. DOI: 10.1109/TSC.2025.3564888.

[12] A. Calagna, Y. Yu, P. Giaccone, and C. F. Chiasserini, "Design, Modeling, and Implementation of Robust Migration of Stateful Edge Microservices," IEEE Transactions on Network and Service Management, vol. 21, no. 2, pp. 1877–1893, 2024. DOI: 10.1109/TNSM.2023.3331750.

[13] P. Bellavista, S. Dahdal, L. Foschini, D. Tazzioli, M. Tortonesi, and R. Venanzi, "Kubernetes Enhanced Stateful Service Migration for ML-Driven Applications in Industry 4.0 Scenarios," in Proc. 2024 IEEE Annual Congress on Artificial Intelligence of Things (AIoT), 2024. DOI: 10.1109/AIoT63253.2024.00015.

[14] L. Poggiani, C. Puliafito, A. Virdis, and E. Mingozzi, "Live Migration of Multi-Container Kubernetes Pods in Multi-Cluster Serverless Edge Systems," in Proc. 1st ACM Workshop on Serverless at the Edge (SEATED '24), pp. 7–12, 2024. DOI: 10.1145/3660319.3660330.